CNN-LSTM Stock Prediction Experiment Report

Reference architecture, 10-epoch results, and diagnostic next steps

Prepared for a GitHub research record. Results were supplied by the experimenter; no training was run and no implementation code was reviewed for this report.

01 Reference model architecture

Based on Abdullah and Salah (2023), A novel hybrid deep learning model for price prediction. The configuration below follows Tables 1 and 2 on page 3425. [1]

Layer (in paper order)	Configuration
Conv1D	Kernel size = 1; filters = 64; activation = ReLU
MaxPooling	Pool size = 2
Dropout 1	Rate = 0.2
Flatten	No additional configuration listed
LSTM 1, then Dropout 2	100 units; return_sequences = True; subsequent dropout rate = 0.2
LSTM 2, then Dropout 3	100 units; return_sequences = True; subsequent dropout rate = 0.2
LSTM 3, then Dropout 4	100 units; subsequent dropout rate = 0.2; return_sequences not specified in the table
Fully connected	1 unit

02 Paper hyperparameters and experiment differences

Parameter	Paper setting
Optimizer / initial learning rate	Adam / 0.0001
Decay rate / loss function	0.6 / MAE
Batch size / epochs	64 / 100
Early stopping	monitor = loss; patience = 10

Confirmed for this experiment: a 42-step input window and 10 training epochs. The remaining settings above are reference values and have not been checked against the implementation. The table does not define the complete decay schedule; a decay rate of 0.6 should not automatically be interpreted as multiplying the learning rate by 0.6 every epoch.

Results after 10 epochs

Main observation: the 42-step CNN-LSTM achieves only a small MAE reduction relative to predictions fixed at zero. These preliminary results do not establish convergence, an implementation bug, or an unsuitable architecture.

03 Original training and evaluation summary

Logged metric	Reported value
Training average loss	0.0155913291491949
Evaluation average loss	0.015539737307259716
Zero baseline MAE (summary log)	0.015564425848424435

04 Full evaluation statistics

Full Validation Results	Reported value
Prediction mean	-0.0002733734145294875
Prediction std	0.0012823045253753662
Target mean	0.0003973348648287356
Target std	0.023427896201610565
Model MAE	0.015539738349616528
Zero baseline MAE	0.015564427711069584
MAE improvement (original report)	0.15862685441970825%
Prediction-target correlation	0.05614299327135086

Relative improvement = (MAE_zero - MAE_model) / MAE_zero x 100, approximately 0.1586%. Using the full evaluation values, the absolute MAE reduction is about 0.00002469, and prediction std / target std is about 5.47%. Minor last-digit differences between logs are preserved as supplied.

05 Interpretation and limits

Predictions are much less dispersed than targets. This is consistent with conservative predictions under a weak signal, but does not alone establish output collapse. The correlation of approximately 0.056 indicates a weak positive association. Its calculation method was not supplied, so it should not be labeled daily Rank IC or treated as evidence of trading performance.

The experimenter also described using the same data for training and evaluation. It is unclear whether the statistics above come from that diagnostic run. If they do, they measure fitting ability rather than independent validation performance. A one-step model was reported to offer limited benefit, but no quantitative comparison was supplied.

Potential causes and hypotheses

The following preserves the four proposed explanations and adds ways to test them. None has been confirmed as the cause of the reported results.

06 Training may be insufficient

This experiment used 10 epochs, compared with the paper's setting of 100. Fewer epochs do not by themselves establish underfitting. Inspect training and validation curves, actual learning rates, and gradients before extending training to 30, 50, and 100 epochs and comparing the best validation checkpoints.

Averaging losses over several days changes batch composition or sample weighting; it does not automatically teach longer-term behavior. Gradient accumulation requires correctly scaled losses and valid gradient paths. Averaging detached numeric logs cannot support backpropagation.

07 A shared model may miss stock-specific behavior

Stocks may differ in their features and scales, but a shared model can learn common patterns from more observations. Separate models have fewer samples per stock and are not necessarily better. Compare a shared model, a shared model with stock or sector identifiers, and stock-specific or grouped models using identical date splits.

08 MAE may be misaligned with the ranking objective

The theoretically optimal prediction under MAE is the conditional median, not the mean. If that median is near zero, near-zero predictions can achieve reasonable MAE. This does not mean MAE forces every output to zero or that the resulting rankings contain no information.

For the JPX ranking task, consider daily cross-sectional Rank IC and the competition-defined Sharpe evaluation. Rank IC typically measures Spearman correlation between predictions and targets across stocks on the same day. A single correlation pooled across dates is not a substitute.

Hard ranks have zero or undefined derivatives. Training on rank correlation therefore requires a differentiable soft-rank approximation or another ranking surrogate. [4] Batches should support comparisons within the same day. Whether improved Rank IC translates into improved Sharpe must be validated separately.

09 The implementation or data pipeline may be flawed

Priority checks include window-label alignment, stock boundaries, inverse scaling, and tensor shapes. A standard Flatten layer removes the time dimension, whereas Keras LSTM expects a three-dimensional sequence input. [2,3] Clarify any TimeDistributed wrapper, subsequence construction, or reshape between these layers; the table alone is insufficient to establish an exact reproduction.

Also check the axis along which the kernel-size-1 convolution operates. Along the time axis, a single such convolution does not combine adjacent dates. This is an architectural property, not inherently a coding error.

Next steps and references

10 Recommended diagnostic sequence

Priority	Action and purpose
1. Small-sample fit	Fix a small set of windows, temporarily remove dropout and other regularization, and train repeatedly. If loss does not decrease substantially, inspect gradients, parameter updates, and input-label alignment.
2. Data and shapes	Log input/output shapes for Conv1D, Pooling, Flatten, and LSTM. Ensure each window contains one stock, is date-sorted, and uses only information available at prediction time.
3. Valid evaluation	Split training and future validation periods by date. Fit scalers only on training data and check whether label intervals cross split boundaries. Record dates, sample counts, features, and the target definition.
4. Longer training	Keep the 10-epoch run as a baseline. Save learning curves and the best checkpoint during longer runs. If stopping on a validation metric, document the change from the paper's monitor=loss.
5. Fair comparisons	Add a constant training-median baseline and a simple model. Then compare shared/grouped models and MAE/ranking losses using fixed splits and multiple random seeds.

For each comparison, retain MAE, prediction standard deviation, the mean and variability of daily Rank IC, and the JPX score computed using its official definition. Account for temporal dependence in statistics across days. No daily results or repeated runs were supplied, so the stability of the 0.159% improvement remains unknown.

11 Sources and reproducibility scope

The architecture tables were checked against the supplied text and the indexed journal content for Tables 1 and 2. The direct journal download did not provide a readable PDF during preparation, so the full paper and architecture figure were not verified page by page. All experiment values were supplied by the experimenter; diagnoses and next steps are analytical recommendations.

[1] Abdullah, W., & Salah, A. (2023). A novel hybrid deep learning model for price prediction. IJECE, 13(3), 3420-3431. Tables 1-2, p. 3425. DOI: 10.11591/ijece.v13i3.pp3420-3431. Supplied paper link.

[2] Keras documentation. Flatten layer.

[3] Keras documentation. LSTM layer.

[4] Blondel, M., Teboul, O., Berthet, Q., & Djolonga, J. (2020). Fast Differentiable Sorting and Ranking. ICML, PMLR 119, 950-959.

Evaluation reference: JPX Tokyo Stock Exchange Prediction - Evaluation. Verify the official formula when implementing evaluation; the competition scoring code was not reproduced for this report.